

VITyarthi Project Report

SMART STUDENT EXPENSE & BUDGET ANALYZER

Using Python and Fundamental Algorithms

Submitted by

Shabd Mathur

Registration No.: 26BAI10433

Course: Introduction to Problem Solving and Programming

Course Code: CSE1021

Faculty: M DARSHAN TEJA

Semester: Fall Sem 2026-27

Academic Year 2026–27

Table of Contents

Section	Topic
1.	Introduction
2.	Problem Statement
3.	Objectives
4.	Functional Requirements
5.	Non-Functional Requirements
6.	System Architecture
7.	Design Diagrams
8.	Design Decisions and Rationale
9.	Implementation Details
10.	Screenshots and Results
11.	Testing Approach
12.	Challenges Faced
13.	Learnings and Key Takeaways
14.	Future Enhancements
15.	Conclusion
16.	References

1. Introduction

Managing personal expenses can become difficult when records are maintained manually. Students may spend money on food, transport, study materials and entertainment without keeping a clear summary of where the money is going.

The Smart Student Expense & Budget Analyzer is a command-line Python application developed to record student expenses and perform basic financial analysis. The project applies fundamental programming concepts such as functions, modules, loops, conditional statements, lists, dictionaries, summation, counting, searching, maximum and minimum finding.

The system follows a modular design so that expense management, budget management, analytics, financial goals, income management, report generation, file handling and validation are separated into individual Python modules.

2. Problem Statement

Students need a simple way to maintain expense information and obtain useful summaries without manually calculating every result. The problem addressed by this project is to design and implement a small command-line system that can manage personal financial records and analyze spending using fundamental Python programming concepts.

The system is intended as an academic project demonstrating problem solving and algorithmic thinking rather than as a production banking or accounting system.

3. Objectives

- Design a modular command-line application using top-down problem solving.
- Record, view, search, filter, update and delete expense records.
- Set monthly and category budgets.
- Apply fundamental algorithms to real-world expense data.
- Calculate total, average, highest and lowest spending.
- Analyze spending by category.
- Track income and financial savings goals.
- Generate a readable final financial report.
- Verify important calculations using automated tests.

4. Functional Requirements

ID	Function	Description
FR1	Add Expense	Create a new expense record with amount, category, description and date.
FR2	View Expenses	Display all stored expense records.
FR3	Search Expenses	Find expenses using a search word in the category or description.
FR4	Filter Expenses	Display expenses belonging to a selected category.
FR5	Edit Expense	Modify an existing expense record.
FR6	Delete Expense	Remove an expense record.
FR7	Budget Management	Set monthly and category budgets and compare spending with them.
FR8	Analytics	Calculate total, average, highest, lowest and category-wise spending.
FR9	Financial Goals	Create goals, view goals, add savings and delete goals.
FR10	Income Management	Store and retrieve the user's income.
FR11	Final Report	Generate a combined summary of income, spending, budgets and goals.
FR12	Testing	Run automated tests for important calculations and project operations.

5. Non-Functional Requirements

Requirement	Description
Usability	The system should provide a clear menu-driven command-line interface.
Input Validation	Invalid or unsuitable values should be handled by basic validation functions.
Maintainability	Separate modules should make individual functions easier to understand and modify.
Performance	Simple algorithms should process normal student expense records quickly.
Reliability	Important data should be saved to JSON files so that it remains available after the program closes.
Portability	The project should run from a terminal on a system with Python 3 installed.

6. System Architecture

The diagram shows the overall structure of the application, from the student/user interface through the main program and functional modules to local JSON data storage.

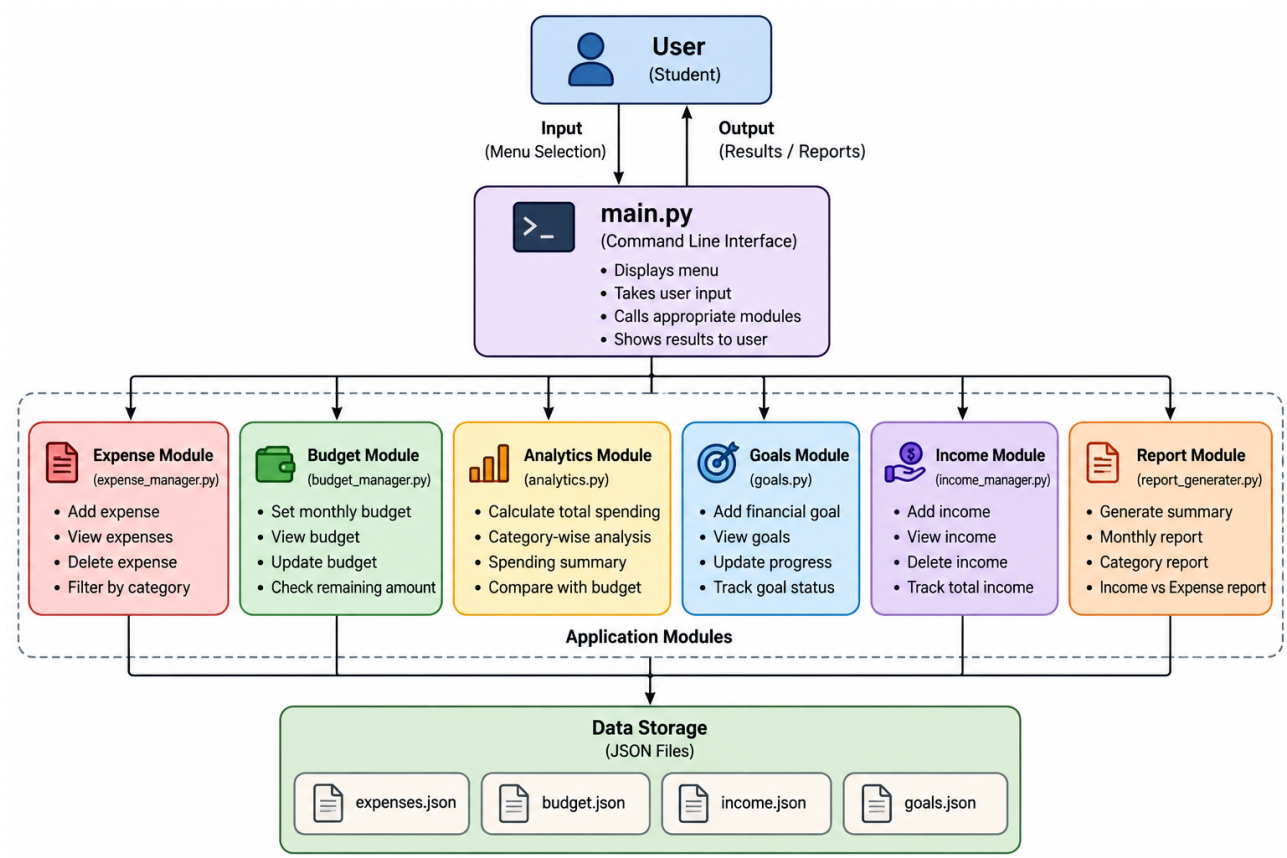


Figure 1. System Architecture of the Smart Student Expense & Budget Analyzer

7.1 Use Case Diagram

The use case diagram shows the major operations available to the student, including expense management, budget management, analytics, financial goals, income management, report generation and testing.

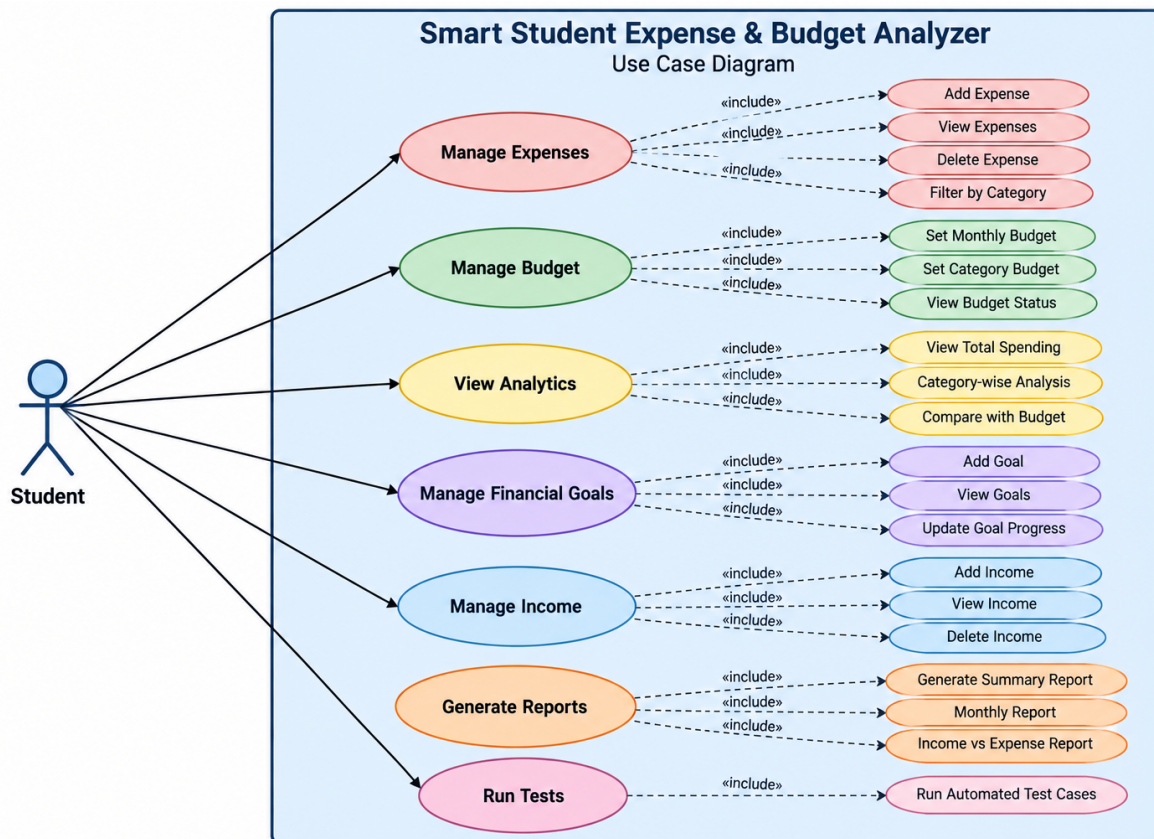


Figure 2. Use Case Diagram

7.2 Workflow Diagram

The workflow describes the normal execution of the command-line application, including menu selection, module processing, result display and returning to the main menu.

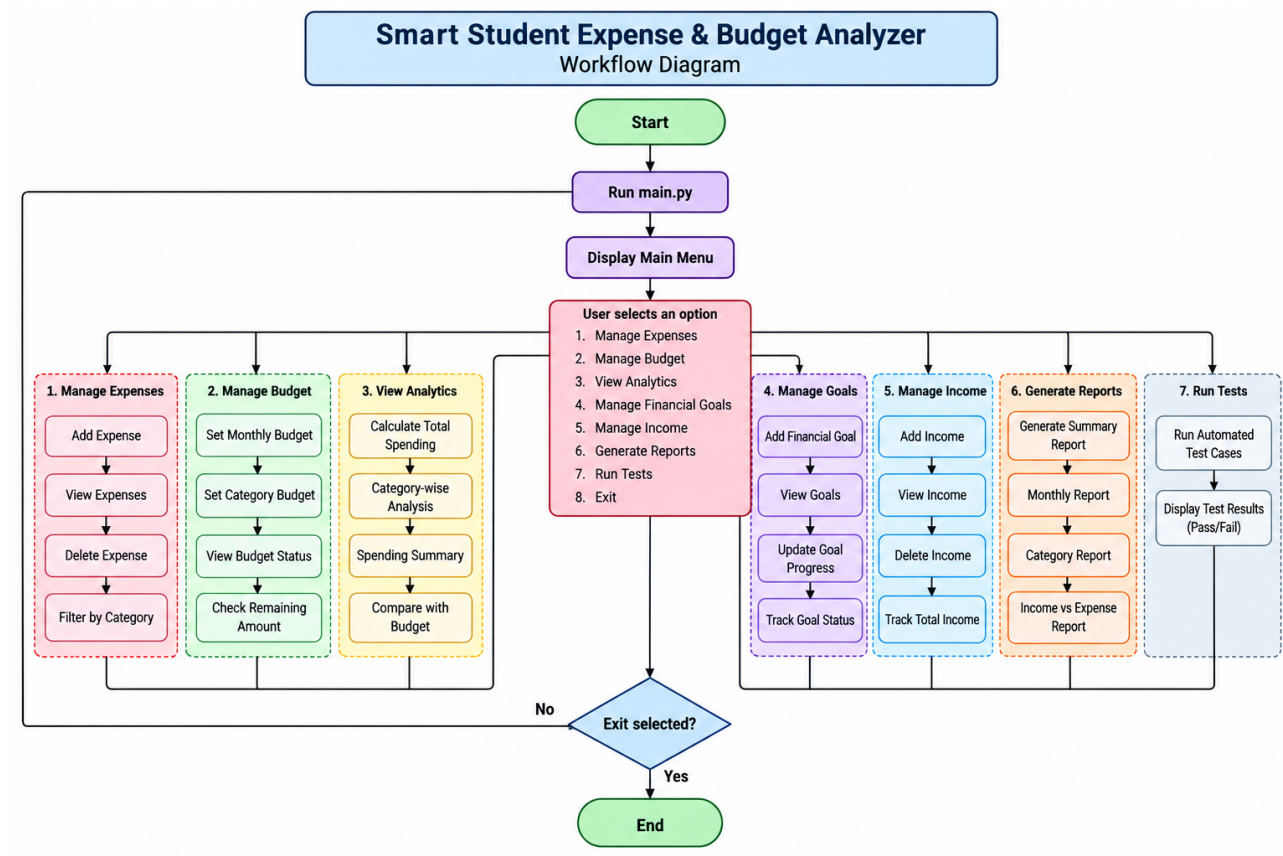


Figure 3. Workflow Diagram

7.3 Sequence Diagram

The sequence diagram illustrates the interaction between the user, main.py, validation, feature modules, JSON storage and report/output during a typical operation.

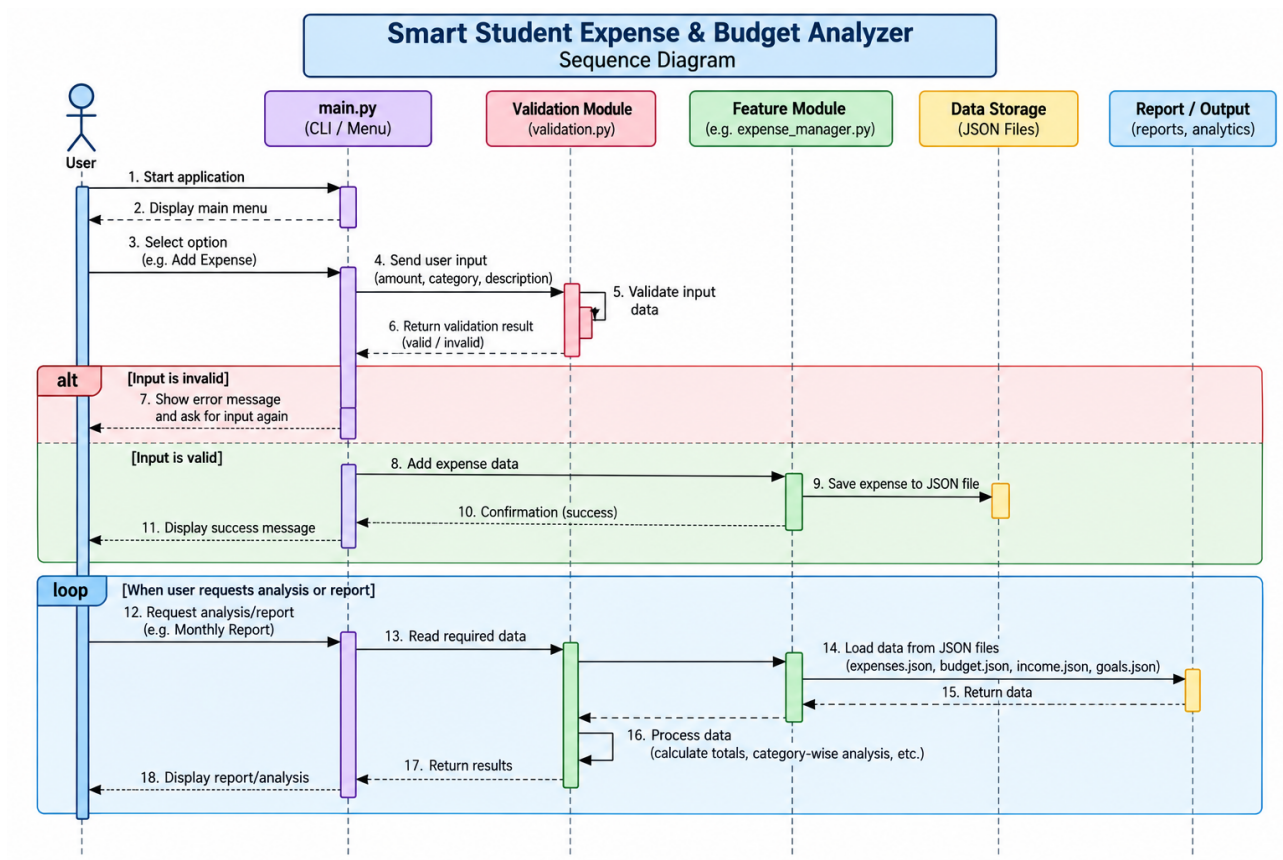


Figure 4. Sequence Diagram

7.4 Class / Component Diagram

The supplied diagram presents the main project modules and their relationships, followed by the system component view showing the presentation, application and data layers.

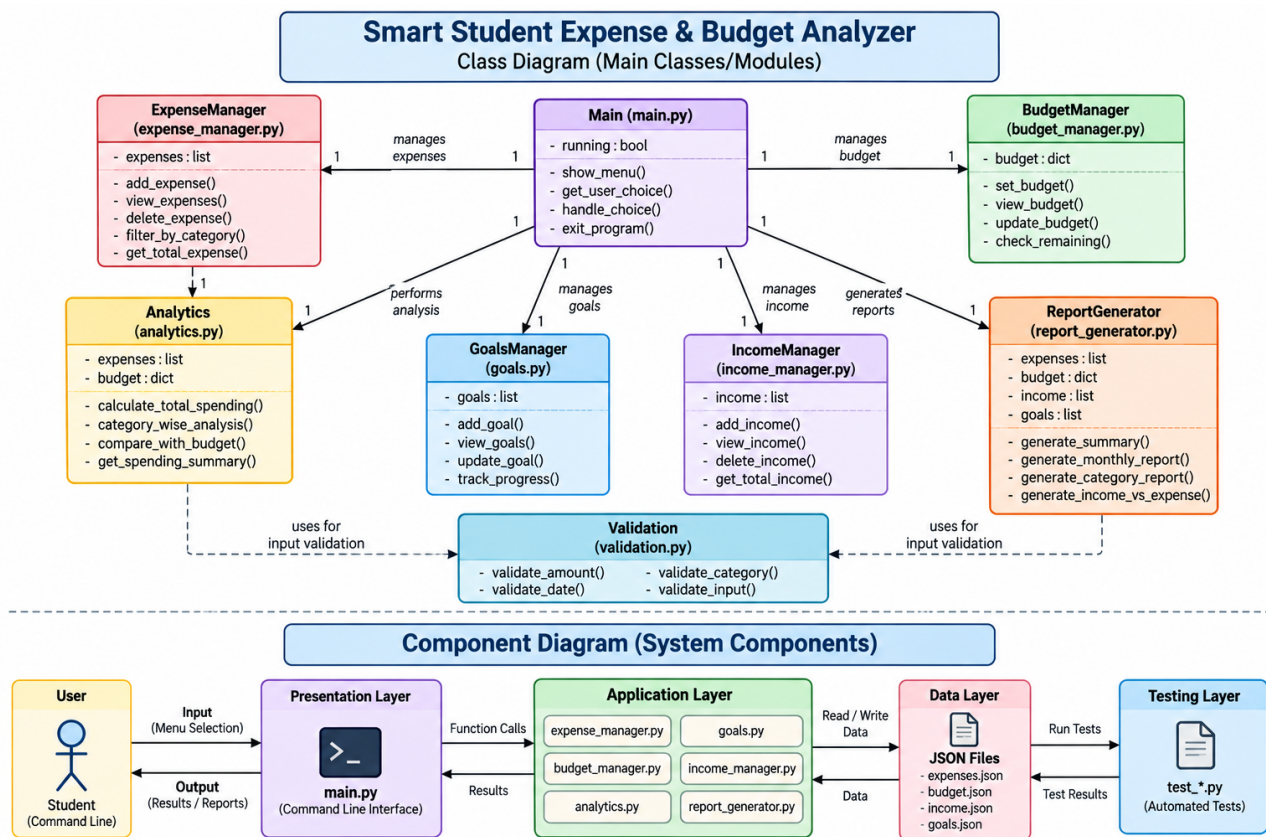


Figure 5. Class / Component Diagram

7.5 Storage / Data Design

The project uses JSON files instead of a database. Each file contains data for one major part of the application.

File	Stored Information
expenses.json	Expense ID, amount, category, description and date
budget.json	Monthly budget and category budgets
income.json	Income amount
goals.json	Goal ID, name, target and saved amount

9. Implementation Details

Module	Responsibility
main.py	Controls the menu and program flow.
expense_manager.py	Adds, displays, searches, filters, edits and deletes expenses.
budget_manager.py	Stores budgets and calculates budget status.
analytics.py	Calculates total, average, highest, lowest and category totals.
goal_manager.py	Creates and manages savings goals.
income_manager.py	Stores and reads income.
report_generator.py	Combines project information into the final report.
file_handler.py	Reads and writes JSON files.
validation.py	Performs basic amount and category validation.

9.1 Data Structures Used

- List: stores multiple expense or goal records.
- Dictionary: represents one expense, goal or budget and stores related fields.
- Nested dictionary: stores category budgets and category spending information.

9.2 Fundamental Algorithms Used

- Summation for total spending.
- Average calculation using total divided by number of records.
- Counting using list length.
- Maximum and minimum finding by comparing values in a loop.
- Linear search for expense descriptions/categories.
- Category-wise accumulation using a dictionary.

10. Screenshots and Results

The following results represent the working project data used during testing and demonstration. Screenshots can be added to this section if required by the final submission format.

10.1 Main Program

The application starts with a menu containing Expense Management, Budget Management, Analytics, Financial Goals, Income Management, Final Report and Exit.

10.2 Sample Final Report

Result	Value
Income	■25,000
Total spending	■850
Remaining money	■24,150
Average expense	■212.50
Number of transactions	4
Highest expense	■350 — Study Notebook
Lowest expense	■120 — Transport Cab
Highest category	Study — ■350
Monthly budget	■15,000
Budget spent	■850
Budget remaining	■14,150
Budget status	Within limit

10.3 Category Analysis

Category	Amount
Food	■180
Transport	■120
Study	■350
Entertainment	■200

10.4 Financial Goal

Goal: New Laptop | Target: ■60,000 | Saved: ■15,000 | Progress: 25% | Status: In Progress.

11. Testing Approach

The project includes a tests folder covering expense operations, budget calculations, analytics, financial goals, income and report calculations. The tests are executed from the terminal using Python's unittest framework.

Test Area	Purpose	Result
Expense Operations	Checks basic expense add/search/filter/edit/delete behavior.	Passed
Budget Calculations	Checks remaining budget, warning and exceeded conditions.	Passed
Analytics	Checks total, average, highest, lowest, category totals and count.	Passed
Financial Goals	Checks goal creation, savings and deletion.	Passed
Income	Checks storing and reading income.	Passed
Report	Checks final report calculations.	Passed

Final test execution: 13 tests ran successfully and all 13 tests passed.

The project was also checked for Python syntax before packaging.

12. Challenges Faced

- Dividing the application into modules while keeping each module simple.
- Designing JSON data structures for different types of information.
- Connecting the menu system with functions from different modules.
- Implementing fundamental algorithms explicitly using loops.
- Testing different operations and checking that the command-line application remains executable.

13. Learnings and Key Takeaways

- Applied top-down design to break a real-world problem into smaller programming tasks.
- Improved understanding of Python functions, modules, loops and conditional statements.
- Practiced lists and dictionaries in a practical application.
- Applied summation, counting, searching and maximum/minimum algorithms.
- Learned basic file handling and JSON data storage.
- Learned how separate Python files can work together as a modular program.
- Practiced testing and debugging a Python project.

14. Future Enhancements

- Add graphical charts for spending categories.
- Add monthly comparison of expenses.
- Add stronger input validation.
- Add more detailed savings progress calculations.
- Move to a database if the amount of data becomes large.
- Add export options for reports.

15. Conclusion

The Smart Student Expense & Budget Analyzer demonstrates how fundamental programming concepts and algorithms can be combined to solve a practical student problem. The application provides expense management, budget tracking, analytics, income management, financial goals and report generation through a modular command-line interface.

The project also demonstrates practical use of top-down design, Python lists and dictionaries, functions, modules, file handling and fundamental algorithms. The sample data and automated tests provide repeatable results for demonstration and verification.

16. References

- Python 3 Documentation — <https://docs.python.org/3/>
- VITyarthi — Build Your Own Project instruction document.
- CSE1021 Introduction to Problem Solving and Programming course syllabus.